Cloud Networking

Appunti - Developing REST APIs With AWS

1. Mappa del modulo

Blocco	Argomenti principali	Perche e importante
API e applicazioni web	API, richieste, risposte, status code	Capire come client, server e database comunicano in una cloud application.
Amazon API Gateway	Gateway come proxy, sicurezza, routing, throttling, monitoraggio	Separare il punto di accesso pubblico dalla logica backend e ridurre il carico sui server.
REST e WebSocket	Request/response stateless vs canale bidirezionale stateful	Scegliere il protocollo corretto in base al tipo di interazione.
Metodi HTTP	GET, POST, PUT, DELETE; safe e idempotent	Progettare endpoint coerenti e prevedibili.
Deployment API	Stage, stage variables, canary deployments	Gestire ambienti dev/prod, versioni e rilasci progressivi.
Lab 6.1	Sito cafe, API Gateway, mock endpoint, DynamoDB	Vedere il ruolo pratico di una REST API in un'architettura cloud.

2. Che cos'e una API

Definizione. Una API, cioe Application Programming Interface, e un meccanismo software che stabilisce come due parti di un sistema comunicano. Nel contesto delle slide, una API semplifica lo sviluppo perche nasconde i dettagli implementativi, espone solo gli oggetti o le azioni necessari allo sviluppatore e definisce il contratto di comunicazione tra chi fornisce informazione e chi la usa.

Idea chiave. Il client non deve sapere come e fatto internamente il database o il server: deve solo conoscere endpoint, metodo, parametri, formato della richiesta e formato della risposta.

Elemento	Significato	Esempio
Client	Applicazione o dispositivo che invia la richiesta.	Browser, app mobile, dashboard, dispositivo IoT.
API	Interfaccia esposta dal server per accedere a risorse o azioni.	GET /products, POST /create_report.
Backend	Sistema che esegue la logica applicativa o recupera dati.	Web server, funzione Lambda, servizio AWS.
Database	Archivio dei dati applicativi.	DynamoDB, RDS, database SQL o NoSQL.
Response	Risposta con status code e, se necessario, dati.	200 OK con JSON, 201 Created, 500 Internal Server Error.

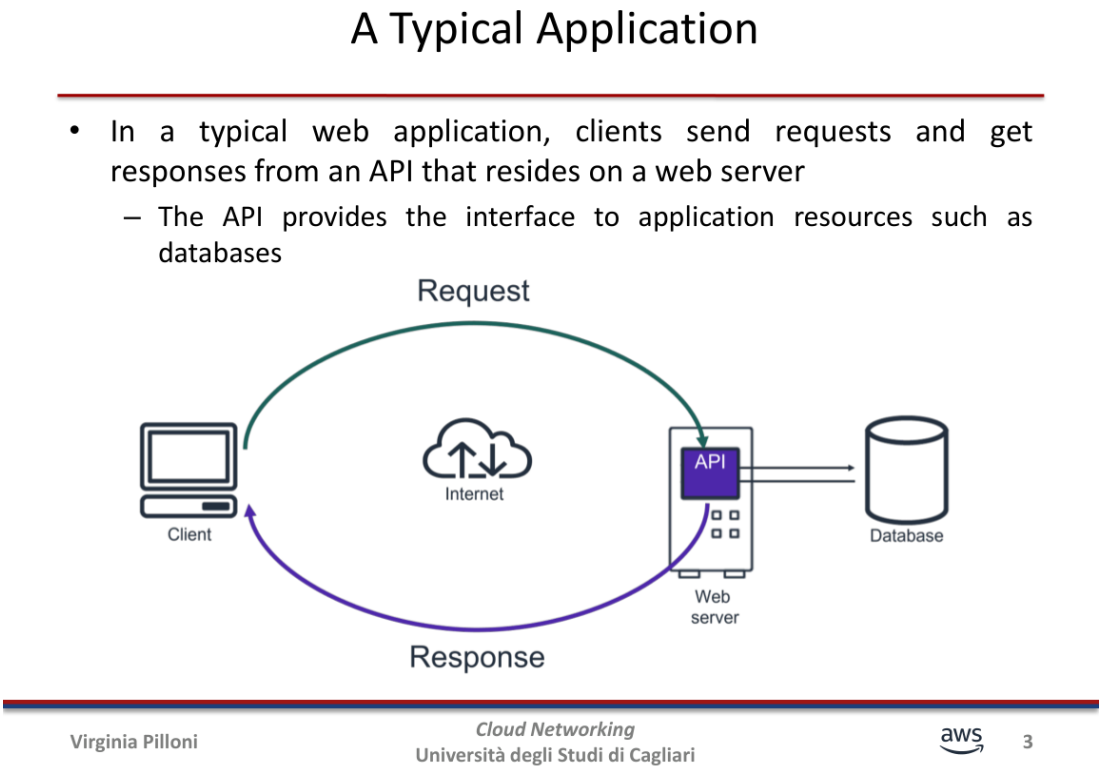


Figura - slide 3: applicazione tipica: il client invia una request, il server espone una API e la API accede al database.

3. Richieste, risposte e status code

Flusso base. Una request parte dal client, attraversa la rete, raggiunge un server o un gateway, viene elaborata e produce una response. La response contiene almeno uno status code; spesso contiene anche dati, tipicamente in formato JSON nelle API REST moderne.

Categoria	Significato	Esempi dalle slide	Interpretazione pratica
20x	Successo	200, 201	La richiesta e stata gestita correttamente. 200 indica risposta OK; 201 indica risorsa creata.
40x	Errore lato client	403, 412	Il client deve correggere qualcosa: autenticazione, autorizzazione, parametri mancanti o precondizioni non rispettate.
50x	Errore lato server	500, 501	Il problema non dipende dal client: errore interno, failure di risorsa, funzionalita non implementata.

Best practice di sicurezza. Le slide evidenziano che, indipendentemente dal protocollo, e buona pratica usare varianti sicure: HTTPS per HTTP e WSS per WebSocket. In un'architettura cloud, questo e fondamentale per proteggere dati, token di autenticazione e richieste applicative.

4. API Gateway: ruolo architetturale

Definizione. Un API Gateway e un proxy tra client e server, oppure tra client e piu server dietro un load balancer. La sua funzione e centralizzare problemi comuni delle applicazioni dinamiche server-driven: controllo del traffico, validazione, routing, sicurezza e riduzione del carico sul backend.

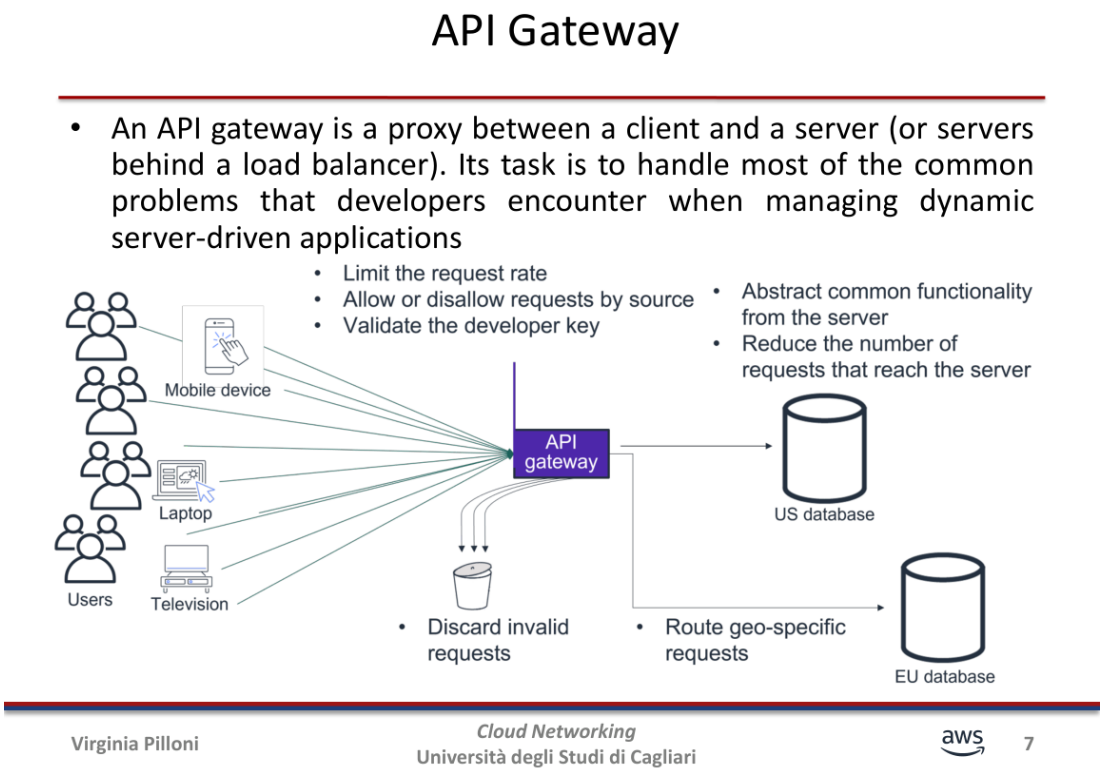


Figura - slide 7: API Gateway come punto di controllo fra utenti/dispositivi e backend differenti.

Funzione del gateway	Cosa fa	Perche serve
Limitare il rate	Riduce o blocca richieste eccessive.	Protegge backend e database da picchi o abuso.
Consentire o negare per sorgente	Applica regole in base alla provenienza.	Aiuta a controllare l'accesso e la superficie esposta.
Validare developer key/token	Controlla credenziali o chiavi API.	Evita accessi non autorizzati.
Scartare richieste invalide	Blocca richieste incomplete o malformate.	Riduce chiamate inutili verso il server.
Routing geo-specifico o verso backend diversi	Indirizza la richiesta al backend corretto.	Supporta scenari multi-backend e multi-area.
Astrarre funzionalita comuni	Sposta logiche ripetitive fuori dal server applicativo.	Semplifica il backend e centralizza policy comuni.

5. Amazon API Gateway

Servizio AWS. Amazon API Gateway e un servizio completamente gestito e serverless per creare, pubblicare, mantenere, monitorare e proteggere API su larga scala. Serverless significa che lo sviluppatore non gestisce direttamente server del gateway: AWS gestisce infrastruttura, disponibilita e scalabilita del servizio.

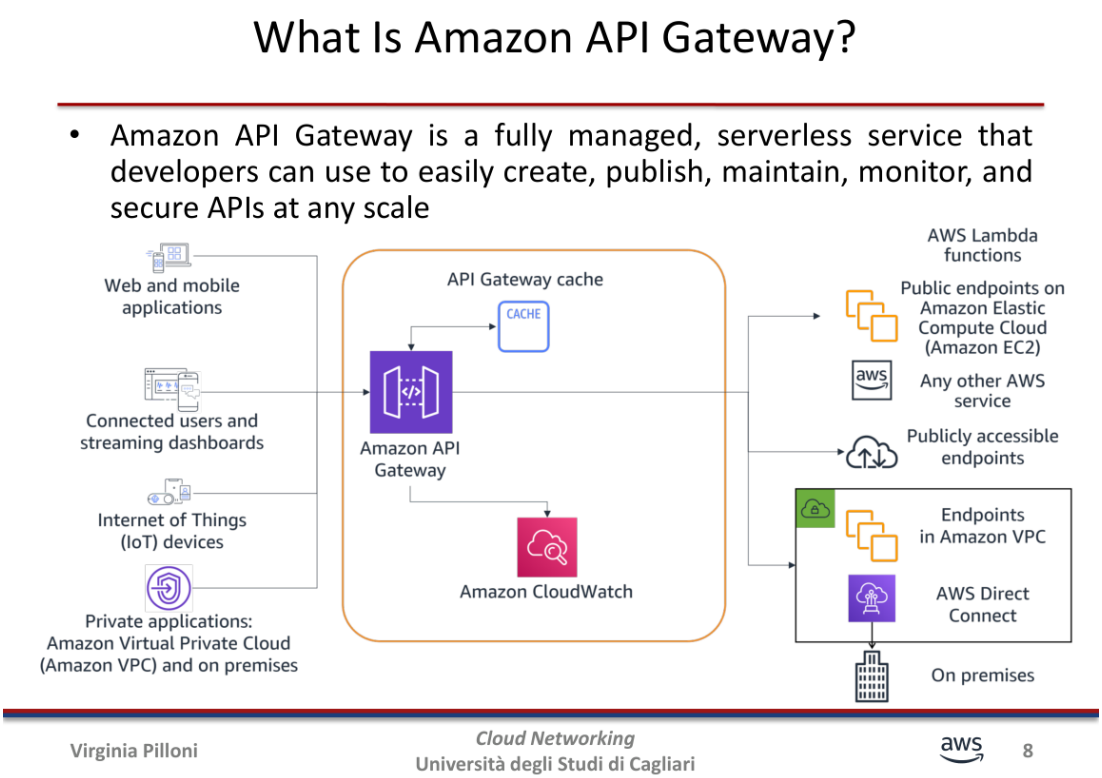


Figura - slide 8: Amazon API Gateway integrato con cache, CloudWatch, Lambda, EC2, servizi AWS, endpoint in VPC, Direct Connect e sistemi on-premises.

Componente/integrabile	Ruolo nell'architettura API Gateway
API Gateway cache	Memorizza risposte frequenti per ridurre latenza e carico sul backend.
Amazon CloudWatch	Raccoglie metriche e supporta monitoraggio, troubleshooting e allarmi.
AWS Lambda	Backend serverless per eseguire logica applicativa a eventi.
Endpoint pubblici su EC2	Backend esposto tramite istanze EC2 o applicazioni su server.
Servizi AWS	Integrazione con servizi gestiti, per esempio DynamoDB o altri servizi applicativi.
Endpoint in Amazon VPC	Accesso a risorse private dentro una rete virtuale.
AWS Direct Connect / on-premises	Integrazione con ambienti aziendali o data center esterni.

6. REST APIs e WebSocket APIs

REST. REST, Representational State Transfer, e uno stile architetturale che usa HTTP come protocollo principale. Il modello tipico e request/response: il client chiama un endpoint, il gateway o il server risponde, la comunicazione e breve e stateless.

WebSocket. WebSocket apre un canale bidirezionale persistente tra client e API. Finche la connessione resta attiva, le informazioni possono fluire in entrambe le direzioni; il modello e quindi adatto a comunicazioni real-time o interattive.

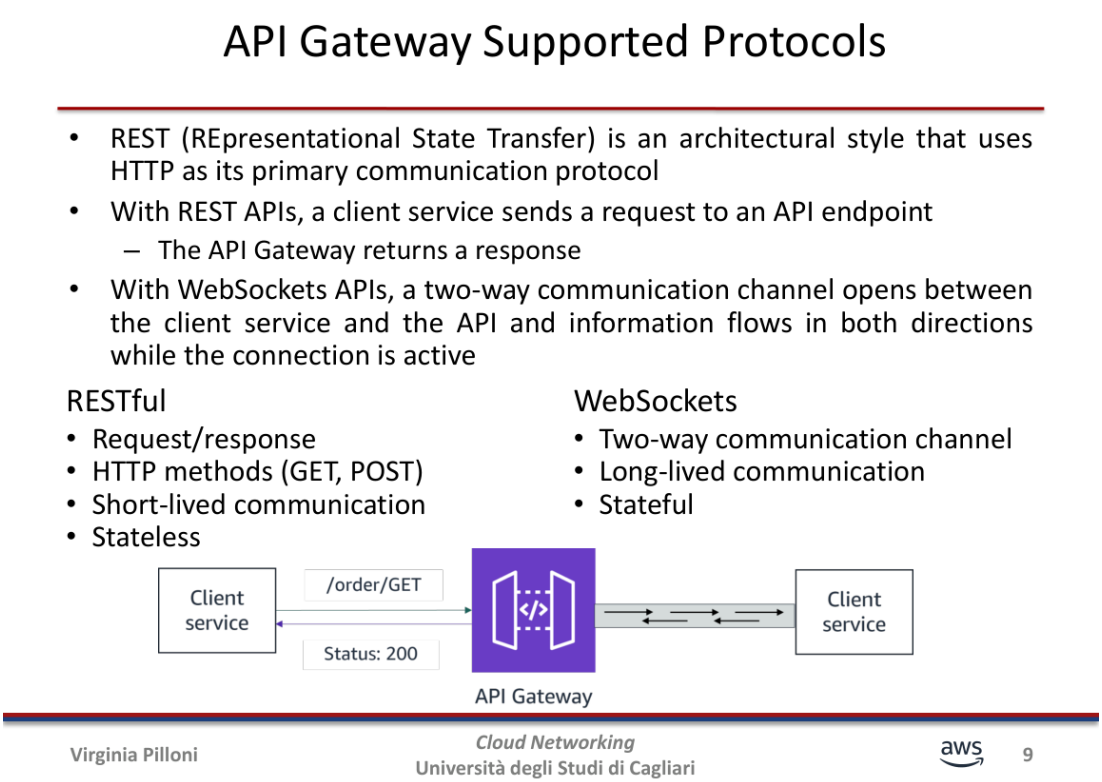


Figura - slide 9: confronto fra RESTful request/response e WebSocket con canale bidirezionale.

Criterio	REST/HTTP API	WebSocket API
Protocollo/URL	http:// oppure https://	ws:// oppure wss://
Tipo di comunicazione	Request/response.	Canale bidirezionale.
Durata	Short-lived: la richiesta nasce e si chiude.	Long-lived: connessione persistente.
Stato	Stateless.	Stateful.
Uso tipico	CRUD, lettura dati, operazioni web classiche.	Chat, notifiche live, dashboard real-time, streaming di eventi.
Overhead	Ogni richiesta porta header e informazioni accessorie.	Dopo apertura connessione si scambiano soprattutto dati.

7. Metodi REST/HTTP principali

I metodi principali evidenziati nelle slide sono GET, POST, PUT e DELETE. La differenza non e solo nominale: cambia la semantica, cioe cosa il metodo promette di fare sulla risorsa.

Metodo	Operazione	Safe?	Idempotent?	Esempio concettuale
GET	Legge il valore di una risorsa.	Si	Si	GET /products oppure GET /products/phone.
POST	Crea una nuova istanza che non ha ancora un proprio URL.	No	No	POST /users con body contenente i dati del nuovo utente.
PUT	Aggiorna una risorsa esistente che ha gia un proprio URL.	No	Si	PUT /users/123 con body aggiornato.
DELETE	Rimuove permanentemente una risorsa.	No	Si	DELETE /users/123.

Safe. Un metodo safe non dovrebbe modificare lo stato della risorsa. GET deve quindi essere usato per leggere, non per creare o cancellare.

Idempotent. Un metodo idempotente produce lo stesso effetto anche se ripetuto piu volte con gli stessi parametri. Per esempio, ripetere PUT /users/123 con lo stesso body lascia la risorsa nello stesso stato finale.

8. Creazione di REST/HTTP API e struttura dell'URL

Creazione. Una REST API o HTTP API puo essere creata tramite AWS CLI oppure tramite console API Gateway. Dopo creazione e deploy, API Gateway genera un URI invocabile dal client.

```
https://{restapi_id}.execute-api.{region}.amazonaws.com/{stage_name}/
```

Parte dell'URL	Significato
{restapi_id}	Identificatore univoco della API.
{region}	Regione AWS in cui la API e stata creata, per esempio us-east-1.
{stage_name}	Nome dello stage di deployment, per esempio dev o prod.

Nota importante. Nelle REST API e WebSocket API lo stage e necessario per rendere l'API accessibile e per distinguere ambienti e versioni. Nelle HTTP API viene creato automaticamente uno stage di default durante la creazione.

9. Creazione di una WebSocket API

Differenza chiave. Per WebSocket APIs, oltre al protocol type, serve una route selection expression. Questa espressione dice ad API Gateway quale campo del messaggio usare per scegliere la rotta applicativa. Nella slide l'esempio usa il campo action del body JSON.

```
aws apigatewayv2 create-api \
  --name "myWebSocketApi" \
  --protocol-type WEBSOCKET \
  --route-selection-expression '$request.body.action'

Esempio messaggio:
{
  "service": "chat",
  "action": "join",
  "data": {
    "room": "room1234"
  }
}
```

10. Deployment, stage e ambienti

Deployment. Dopo aver creato una API, bisogna fare il deployment per renderla accessibile. Il deployment viene associato a uno stage. Uno stage e uno snapshot dell'API identificato da API ID e stage name, ed e incluso nell'URL di invocazione.

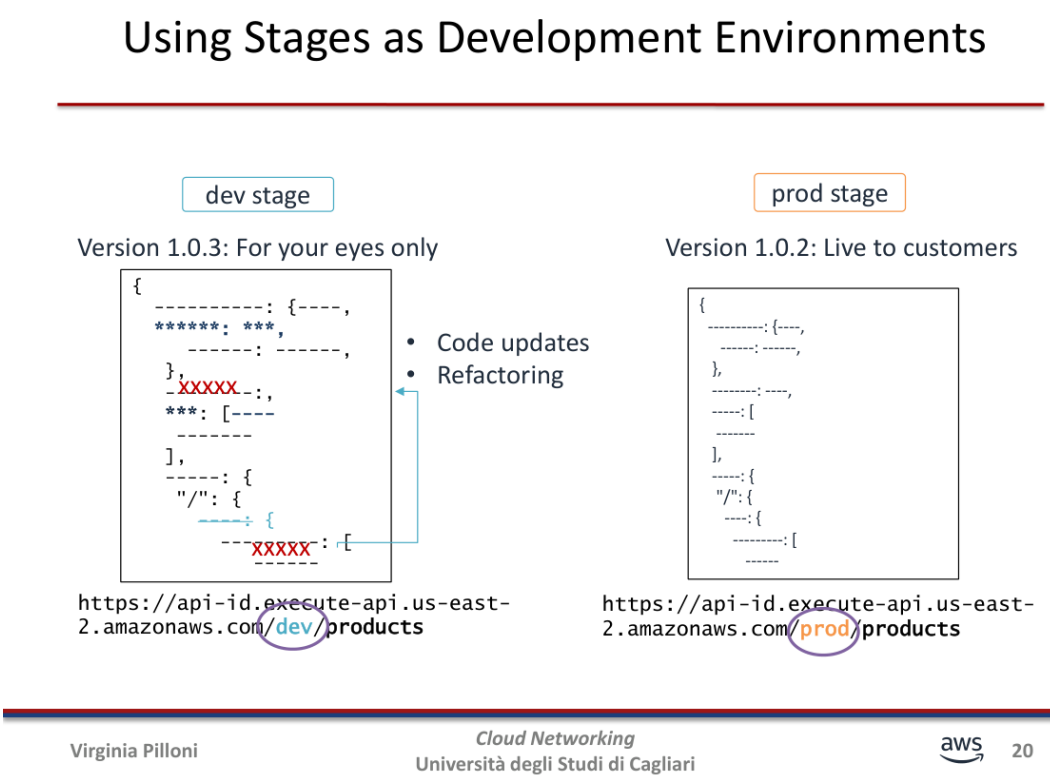


Figura - slide 20: uso di stage diversi come ambienti di sviluppo: dev per test e refactoring, prod per utenti reali.

Uso degli stage	Spiegazione	Esempio
Ambienti diversi	Separano sviluppo, test e produzione.	/dev/products e /prod/products.
Versioning operativo	Permettono di testare una versione prima del rilascio pubblico.	dev con versione 1.0.3, prod con versione 1.0.2.
Backend diversi	Con stage variables uno stage puo puntare a un backend diverso senza cambiare codice applicativo.	dev -> Lambda alias dev; prod -> Lambda alias prod.
Ottimizzazioni per stage	Caching, throttling o configurazioni possono essere applicati in modo diverso per ambiente.	Caching attivo solo in prod.

11. Stage variables

Concetto. Le stage variables sono variabili configurate a livello di stage. Servono a usare valori diversi nei vari ambienti, per esempio collegare la stessa API a una Lambda o a un database diverso in dev, beta e prod, senza modificare il codice applicativo.

Connecting to Different Backends by Using Stage Variables

- Stage-level variables allow the configuration of different values for each stage. This approach enables targeting distinct resources (such as a development database instead of a production database) without requiring changes to the application code

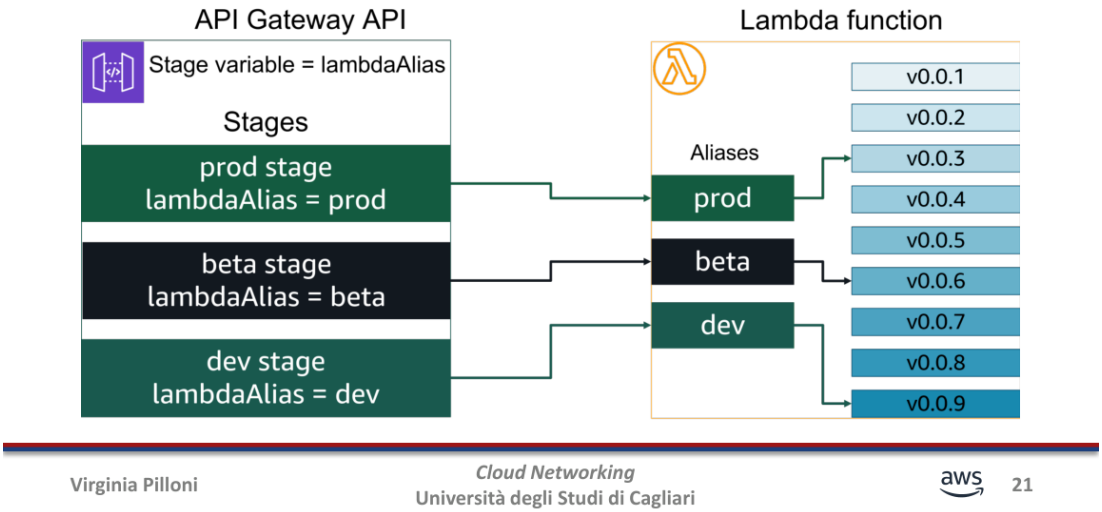


Figura - slide 21: stage variables che puntano a Lambda aliases diversi: prod, beta e dev.

Stage	Variabile esempio	Backend selezionato
prod	lambdaAlias = prod	Alias Lambda stabile per produzione.
beta	lambdaAlias = beta	Versione intermedia per test piu ampi.
dev	lambdaAlias = dev	Versione di sviluppo per modifiche frequenti.

12. Canary deployments

Definizione. Una canary release e una strategia di rilascio in cui una nuova versione della API viene pubblicata accanto alla versione base di produzione sullo stesso stage. Il traffico totale viene diviso casualmente tra produzione e canary secondo una percentuale configurata.

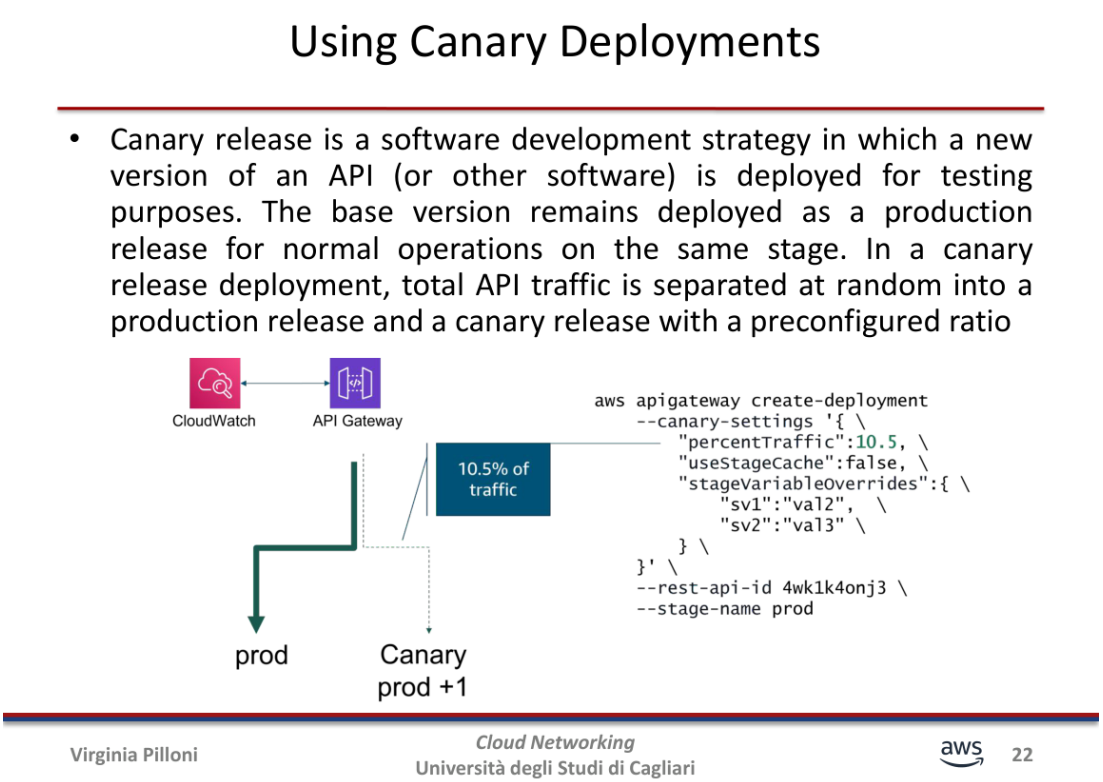


Figura - slide 22: canary deployment: una piccola percentuale di traffico viene inviata alla nuova versione.

Passo	Azione	Motivazione
1	Mantieni la versione prod stabile.	Garantisce continuita per la maggior parte degli utenti.
2	Invia una piccola percentuale al canary.	Riduce il rischio se la nuova versione ha problemi.
3	Monitora metriche ed errori con CloudWatch.	Verifica latenza, errori 4xx/5xx, comportamento del backend.
4	Se tutto va bene, aumenta la percentuale.	Rilascio progressivo e controllato.
5	Promuovi la canary a produzione.	La nuova versione diventa quella principale.

Criticita. Una canary senza monitoraggio non ha senso: se non si osservano metriche, errori e comportamento degli utenti, non si puo decidere se aumentare il traffico o fare rollback.

13. Analisi architetturale: API Gateway in una cloud application

Componenti

- Client: browser, mobile app, dashboard o dispositivo IoT che invia richieste.
- Amazon API Gateway: punto di ingresso gestito, serverless, monitorabile e configurabile.
- Backend: Lambda, EC2, servizio AWS, endpoint VPC, Direct Connect o risorsa on-premises.
- Database: DynamoDB, RDS o altra sorgente dati.
- CloudWatch: metrica e osservabilita della API e dei backend.
- Stage: ambiente logico di deployment, ad esempio dev, beta, prod.

Analisi del flusso

Client -> API Gateway -> validazione/routing/throttling -> backend applicativo -> database o servizio -> risposta -> API Gateway -> client. Il gateway separa il contratto pubblico dell'API dalla logica interna. Questo permette di cambiare backend, versione o configurazione senza esporre direttamente i dettagli implementativi al client.

Identificazione criticita

- Latenza: caching e routing riducono il carico ma una configurazione errata puo aggiungere ritardi.
- Sicurezza: endpoint pubblici senza HTTPS, token o validazione espongono dati e backend.
- Costi: richieste eccessive o canary non controllate possono aumentare consumo e log.
- Versioning: deploy diretto in prod senza stage o canary aumenta il rischio di downtime logico.
- Backend overload: senza throttling, caching o controllo del rate il database puo saturarsi.

Raccomandazioni

- Usare HTTPS/WSS come default.
- Separare dev, beta e prod con stage dedicati.
- Usare stage variables per collegare backend diversi senza cambiare codice.
- Usare canary deployments per rilasci rischiosi o nuove versioni API.
- Monitorare con CloudWatch status code, latenza, errori e traffico.
- Applicare throttling/rate limit quando il backend o il database hanno limiti di capacità.

14. Lab 6.1: Developing REST APIs with Amazon API Gateway

Scenario. Nel laboratorio, Sofia costruisce una web application per un cafe. Esiste una tabella DynamoDB chiamata FoodProducts con informazioni sugli elementi del menu. L'obiettivo e aggiungere la possibilita di interrogare informazioni del menu tramite chiamate REST API. Nello sviluppo iniziale vengono usati mock data endpoints.

Task del laboratorio	Cosa si fa	Obiettivo
1. Preparazione ambiente	Configurazione sviluppo e risorse necessarie.	Avere gli strumenti pronti per creare API e testarle.
2. Primo endpoint GET	Creazione di una route di lettura.	Restituire dati tramite richiesta REST.
3. Secondo endpoint GET	Aggiunta di un altro endpoint di lettura.	Esporre un ulteriore caso di interrogazione.
4. Endpoint POST	Creazione di endpoint per invio dati o generazione report.	Mostrare operazione non di sola lettura.
5. Deploy API	Associazione API a uno stage.	Rendere invocabile la API tramite URL.
6. Aggiornamento sito	Il frontend usa gli endpoint API.	Collegare pagina web e API Gateway.

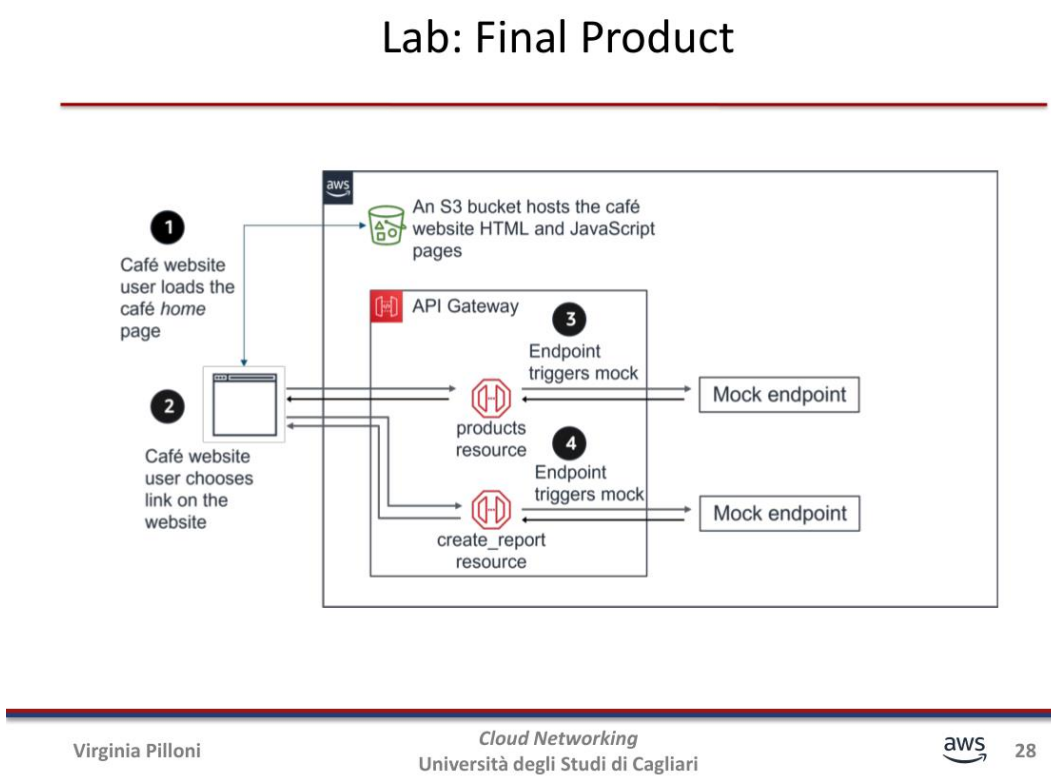


Figura - slide 28: prodotto finale del laboratorio: sito cafe su S3, API Gateway con risorse products e create_report, mock endpoint.

Lettura del diagramma finale. Il sito del cafe e ospitato su S3. L'utente carica la pagina e seleziona link o funzioni dal frontend. Le chiamate vengono indirizzate ad API Gateway, che espone risorse come products e create_report. Gli endpoint, in questa fase iniziale, attivano mock endpoint: questo consente di testare contratto API e integrazione frontend prima di collegare un backend reale.

15. Cose da ricordare per l'esame

- API = contratto di comunicazione: nasconde implementazione, espone azioni/oggetti, stabilisce formato della comunicazione.
- API Gateway = proxy gestito tra client e backend: sicurezza, routing, throttling, monitoraggio, caching e riduzione del carico.
- REST = HTTP, request/response, stateless, comunicazione breve.
- WebSocket = canale bidirezionale, stateful, comunicazione persistente.
- GET e safe e idempotente; POST non e safe e non e idempotente; PUT e DELETE non sono safe ma sono idempotenti.
- Gli stage sono snapshot/deployment environment: dev, beta, prod.
- Le stage variables permettono backend diversi per stage senza modificare il codice.
- Canary deployment = rilascio progressivo con percentuale di traffico verso la nuova versione.
- Le API vanno monitorate: senza CloudWatch non si controllano errori, latenza e comportamento del traffico.

16. Glossario essenziale

Termine	Definizione sintetica
API	Interfaccia che stabilisce come un client interagisce con una risorsa o un servizio.
Endpoint	URL o rotta specifica a cui il client invia la richiesta.
Request	Messaggio inviato dal client, con metodo, path, headers e talvolta body.
Response	Messaggio restituito dal server, con status code e dati opzionali.
REST	Stile architetturale basato su HTTP, risorse e operazioni stateless.
WebSocket	Protocollo per connessione persistente bidirezionale.
API Gateway	Servizio/proxy che gestisce accesso, routing e policy verso backend.
Stage	Ambiente/snapshot di deployment di una API.
Stage variable	Variabile associata a uno stage per cambiare configurazioni o backend.
Canary release	Rilascio progressivo di una nuova versione su una frazione del traffico.
Mock endpoint	Endpoint fittizio usato per testare API e frontend prima del backend reale.
CloudWatch	Servizio AWS per metriche, log, allarmi e monitoraggio.

17. Domande d'esame con risposta

Domanda 1. Che cosa e una API?

Risposta: E una interfaccia software che definisce come un information user e un information provider comunicano, nascondendo i dettagli interni ed esponendo solo azioni o oggetti necessari.

Domanda 2. Qual e il ruolo di una API in una web application tipica?

Risposta: Fornisce al client un punto di accesso controllato alle risorse applicative, come database o servizi backend, tramite richieste e risposte.

Domanda 3. Che cosa contiene una risposta API?

Risposta: Contiene almeno uno status code e, quando necessario, dati restituiti al client, spesso in formato JSON.

Domanda 4. Che cosa indicano gli status code 20x?

Risposta: Indicano risposte di successo, per esempio 200 OK o 201 Created.

Domanda 5. Che cosa indicano gli status code 40x?

Risposta: Indicano che il client deve correggere qualcosa, per esempio credenziali errate, autorizzazione mancante o dati insufficienti.

Domanda 6. Che cosa indicano gli status code 50x?

Risposta: Indicano un problema lato server o backend, non causato direttamente dal client.

Domanda 7. Che cosa e un API Gateway?

Risposta: E un proxy tra client e backend che gestisce funzioni comuni come routing, rate limiting, validazione, sicurezza e riduzione del carico sui server.

Domanda 8. Perche Amazon API Gateway e definito serverless?

Risposta: Perche lo sviluppatore crea e configura API senza gestire direttamente i server che eseguono il gateway; AWS gestisce infrastruttura e scalabilita.

Domanda 9. Qual e la differenza principale tra REST e WebSocket?

Risposta: REST usa un modello request/response breve e stateless; WebSocket usa un canale bidirezionale persistente e stateful.

Domanda 10. Quando useresti REST invece di WebSocket?

Risposta: Userei REST per operazioni CRUD, interrogazioni e servizi stateless. WebSocket e preferibile per comunicazioni real-time persistenti.

Domanda 11. Che cosa significa che GET e safe?

Risposta: Significa che GET deve leggere dati senza modificare lo stato della risorsa sul server.

Domanda 12. Che cosa significa idempotente?

Risposta: Una operazione idempotente produce lo stesso effetto finale anche se ripetuta piu volte con gli stessi parametri.

Domanda 13. POST e idempotente?

Risposta: No. POST crea una nuova istanza e ripetere la stessa richiesta puo creare piu risorse o effetti multipli.

Domanda 14. PUT e idempotente?

Risposta: Si. Ripetere lo stesso PUT sulla stessa risorsa porta allo stesso stato finale.

Domanda 15. DELETE e idempotente?

Risposta: Si dal punto di vista dell'effetto finale: dopo una o piu DELETE, la risorsa risulta eliminata.

Domanda 16. Che cosa e uno stage in API Gateway?

Risposta: E uno snapshot/deployment environment della API, identificato da API ID e stage name, incluso nell'URL di invocazione.

Domanda 17. Perche usare stage dev e prod separati?

Risposta: Per testare modifiche, refactoring o nuove versioni in dev senza impattare gli utenti reali che usano prod.

Domanda 18. A cosa servono le stage variables?

Risposta: Permettono di usare valori diversi per ogni stage, per esempio alias Lambda o database diversi, senza modificare il codice applicativo.

Domanda 19. Che cosa e una canary release?

Risposta: E un rilascio progressivo in cui una parte del traffico viene inviata alla nuova versione mentre la versione stabile continua a servire la maggioranza degli utenti.

Domanda 20. Perche il laboratorio usa mock endpoint?

Risposta: Per testare struttura, rotte e integrazione API/frontend prima di collegare il backend reale o la logica definitiva.

18. Schema finale di ripasso

```
Client
  -> HTTPS request
    -> Amazon API Gateway
       - valida richiesta / chiave / token
       - applica throttling o caching
       - sceglie rotta e stage
       - inoltra al backend
          -> Lambda / EC2 / AWS service / VPC endpoint / on-premises
             -> Database o servizio applicativo
          <- response
    <- status code + dati
<- risposta al client
```

Sintesi progettuale. Per una applicazione cloud moderna, API Gateway e il punto di controllo tra utenti e backend. REST e adatto a interazioni stateless e CRUD; WebSocket e adatto a comunicazioni persistenti. Stage, stage variables e canary deployments servono a governare il ciclo di vita della API senza interrompere il servizio.